

CI/CD with GitHub Actions

Level Up Your Code Game!

Automating Your Software Development Workflow

Navneet K Pandey — Finn.no | April 2026

The Problem

Without CI/CD

- Push code Friday at 5 PM
- Monday: production is on fire
- "Who broke it?" — nobody knows
- Debugging at 3 AM
- Merge conflicts pile up

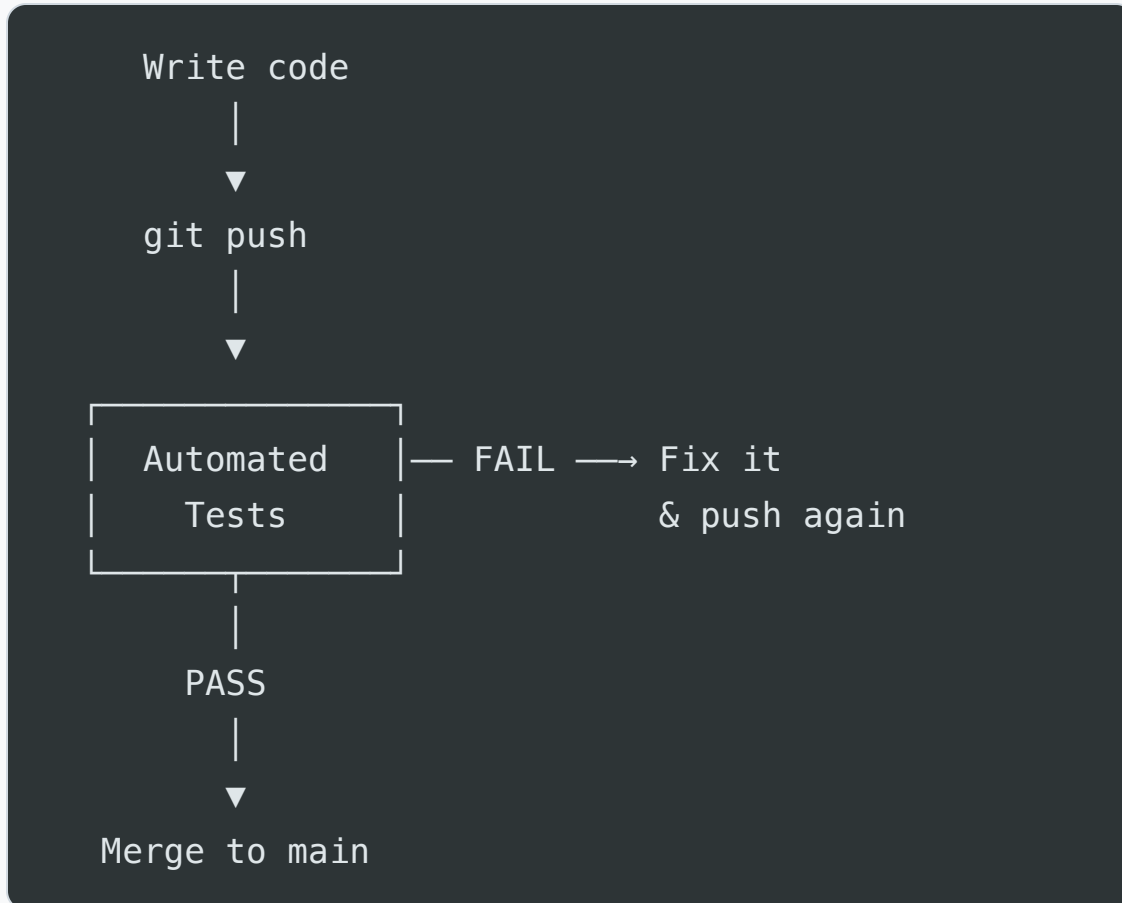
With CI/CD

- Push code anytime
- Tests catch bugs in **2 minutes**
- Git blame shows which commit
- Deploy with confidence
- Small merges = small conflicts

*Without CI/CD, bugs are discovered by **users**. With CI/CD, bugs are discovered by **robots**.*

CI — Continuous Integration

Merge small changes often. Let a robot test them every time.



The key rule:

Integrate small changes often.

Don't hoard code for weeks.

Requirements:

1. A shared repository (GitHub)
2. Frequent commits
3. **Automated tests** ← non-negotiable

CD — Continuous Delivery

The autopilot flies the approach and aligns everything perfectly.

Then it says: **"Ready to land, Captain."**

The pilot decides when to touch down.

Code is always **ready** to deploy. A human presses the button.

```

No automation  →  CI  →  Continuous Delivery  →  Continuous Deployment
                  ↑                ↑
                Most teams      Netflix, GitHub

```

CD — Continuous Deployment

The autopilot flies the approach, checks every parameter, and **lands the plane itself**.

The pilot monitors dashboards and can take over — but doesn't need to.

Code goes to production **automatically** after tests pass. No human gate.

No automation → CI → Continuous Delivery → Continuous Deployment

↑

Requires: high test
confidence, monitoring,
rollback capability

The Autopilot Analogy

Why Continuous Deployment requires **more** discipline, not less:

Autopilot Landing	Continuous Deployment
Checks altitude, speed, alignment	Runs tests, lint, security scans
Lands automatically if all checks pass	Deploys automatically if all checks pass
Pilot monitors dashboards	Team monitors Grafana & alerts
Pilot can take manual control	Team can rollback
Requires extreme system trust	Requires extreme test confidence
Aborts if conditions are wrong	Build fails if tests fail

It's not reckless — it requires **more testing, more monitoring, and better rollback** than manual deployment.

GitHub Actions 101

Your new best friend

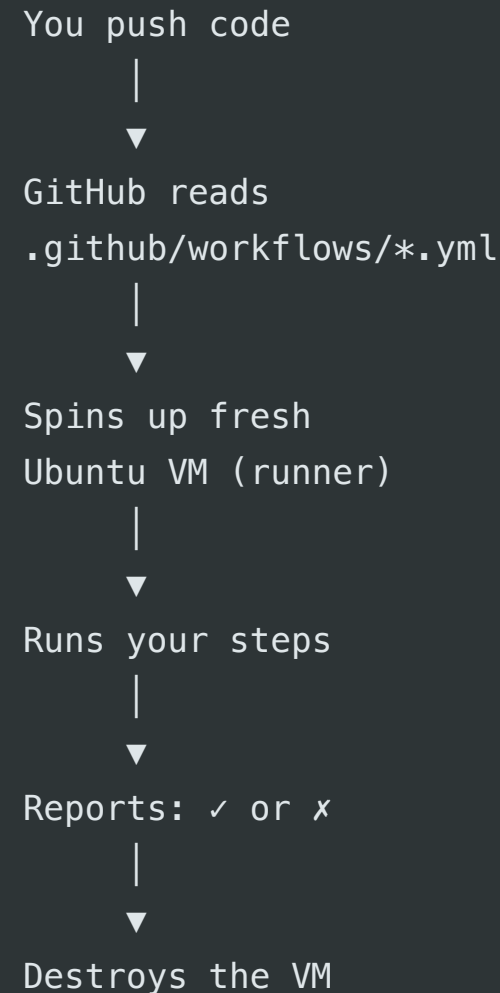
What is GitHub Actions?

GitHub's built-in CI/CD tool. **Free** for public repos.

*A robot that reads your
`.github/workflows/*.yml` files and runs
them when something happens in your
repo.*

The key insight:

GitHub gives you a **fresh virtual machine**, runs your commands, and **throws it away**. Every run starts clean.



```
graph TD; A[You push code] --> B[GitHub reads  
.github/workflows/*.yml]; B --> C[Spins up fresh  
Ubuntu VM (runner)]; C --> D[Runs your steps]; D --> E[Reports: ✓ or ✗]; E --> F[Destroys the VM]
```

You push code
↓
GitHub reads
.github/workflows/*.yml
↓
Spins up fresh
Ubuntu VM (runner)
↓
Runs your steps
↓
Reports: ✓ or ✗
↓
Destroys the VM

Key Concepts

```
WORKFLOW ← the .yaml file itself
├── EVENT   ← "when should this run?" (push, pull_request, schedule)
├── JOBS    ← "what groups of work?" (run in PARALLEL by default)
│   └── STEPS ← "what commands?" (run SEQUENTIALLY)
│       ├── uses: ← pre-built action (someone else wrote it)
│       └── run:  ← shell command (you write it)
```

Jobs run in **parallel** unless
you add **needs**

Steps within a job run
sequentially

uses = import a library
run = write your own code

YAML — The Language of Workflows

Two rules: **spaces not tabs** and **indentation = structure**

```
name: CI                                # Name shown in the Actions tab

on: [push, pull_request]                # When to run (the trigger)

jobs:                                    # What to do
  test:                                  #   Job name (you choose this)
    runs-on: ubuntu-latest               #   Which runner (machine) to use
    steps:                                #   Ordered list of things to do
      - name: Checkout code
        uses: actions/checkout@v6        # Use a pre-built action
      - name: Set up Python
        uses: actions/setup-python@v6
        with:
          python-version: "3.12"
      - name: Install dependencies
        run: pip install -r requirements.txt # Run a shell command
      - name: Run tests
        run: pytest --verbose              # The moment of truth!
```

What Can GitHub Actions Automate?

Testing

Run your test suite on every push. Catch bugs before they reach users.

Code Quality

Linting, formatting, static analysis. Keep the codebase clean.

Build & Deploy

Build Docker images, deploy to cloud, publish packages.

Notifications

Send Slack messages, post PR comments, create issues.

If you can do it in a terminal, GitHub Actions can automate it.

Exercise 1

Your First CI Pipeline — open your laptops!

Exercise 1: Setup

Step 1: Fork the repository

<https://github.com/navneetpandey/ci-cd-lab-2026>

Step 2: Create the workflow file

1. Click **Add file** → **Create new file**
2. Name it: `.github/workflows/ci.yml`
3. Paste the content from `exercises/01-basic-ci.yml`
4. Commit to `main`

Step 3: Watch it run!

Go to the **Actions** tab. See the yellow spinner?

A machine just woke up in a data center somewhere **because you pushed code.**



Scan to open
repo

Exercise 1: Break It!

1. Edit `app/calculator.py` — change `add` to return the wrong value:

```
def add(a: float, b: float) -> float:  
    return a - b # BUG! This should be a + b
```

2. Commit and push → go to **Actions** → watch it **FAIL**

```
FAILED tests/test_calculator.py::test_add_positive_numbers  
    assert add(2, 3) == 5  
AssertionError: assert -1 == 5
```

3. Fix it → change back to `return a + b` → push → green ✓

The CI loop: push → fail → read error → fix → push → pass

Going Deeper

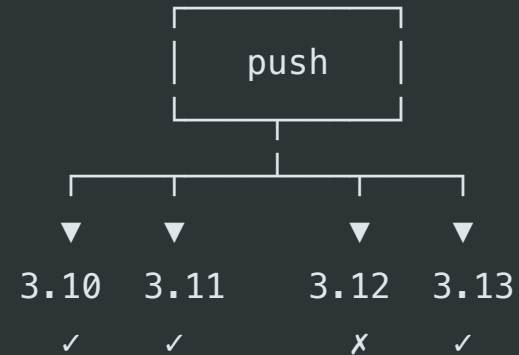
Matrix, caching, and job dependencies

Matrix Strategy — Test on Multiple Versions

One config block. Four parallel jobs.

```
strategy:
  matrix:
    python-version:
      - "3.10"
      - "3.11"
      - "3.12"
      - "3.13"
```

GitHub copies your job 4 times:



All run AT THE SAME TIME.

Real teams test across OS, database versions, and language versions — all in parallel from one config block.

Caching — Speed Up Your Builds

Without Cache

Install deps: 45s x4 runs

Total: 180s

With Cache

First: 45s / Then: 3s x3

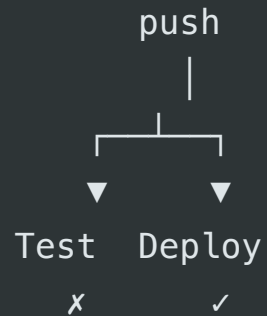
Total: 54s

```
- uses: actions/cache@v5
  with:
    path: ~/.cache/pip
    key: ${{ runner.os }}-pip-${{ hashFiles('requirements.txt') }}
```

`hashFiles` = cache busts automatically when `requirements.txt` changes.

Job Dependencies — **needs**

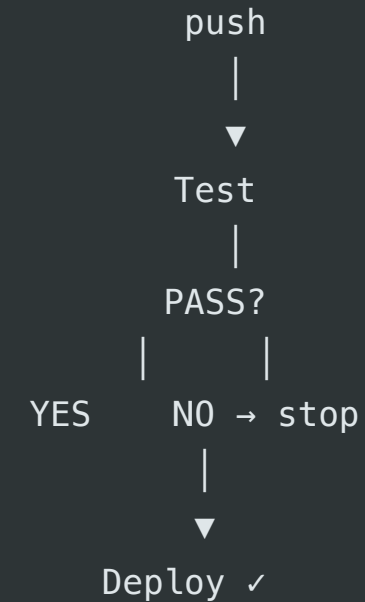
Without **needs**



Deploy ran even though tests failed!

Broken code is live.

With **needs**



Deploy only if tests pass.

```
deploy:
  needs: [test, lint]    # Both must pass before deploy starts
```

Branch Protection

Feature branches get tested. Main branch gets deployed.

Push to feature branch

- Test  Lint  Build 
- Deploy  **SKIPPED**

"Not main branch"

Push to main

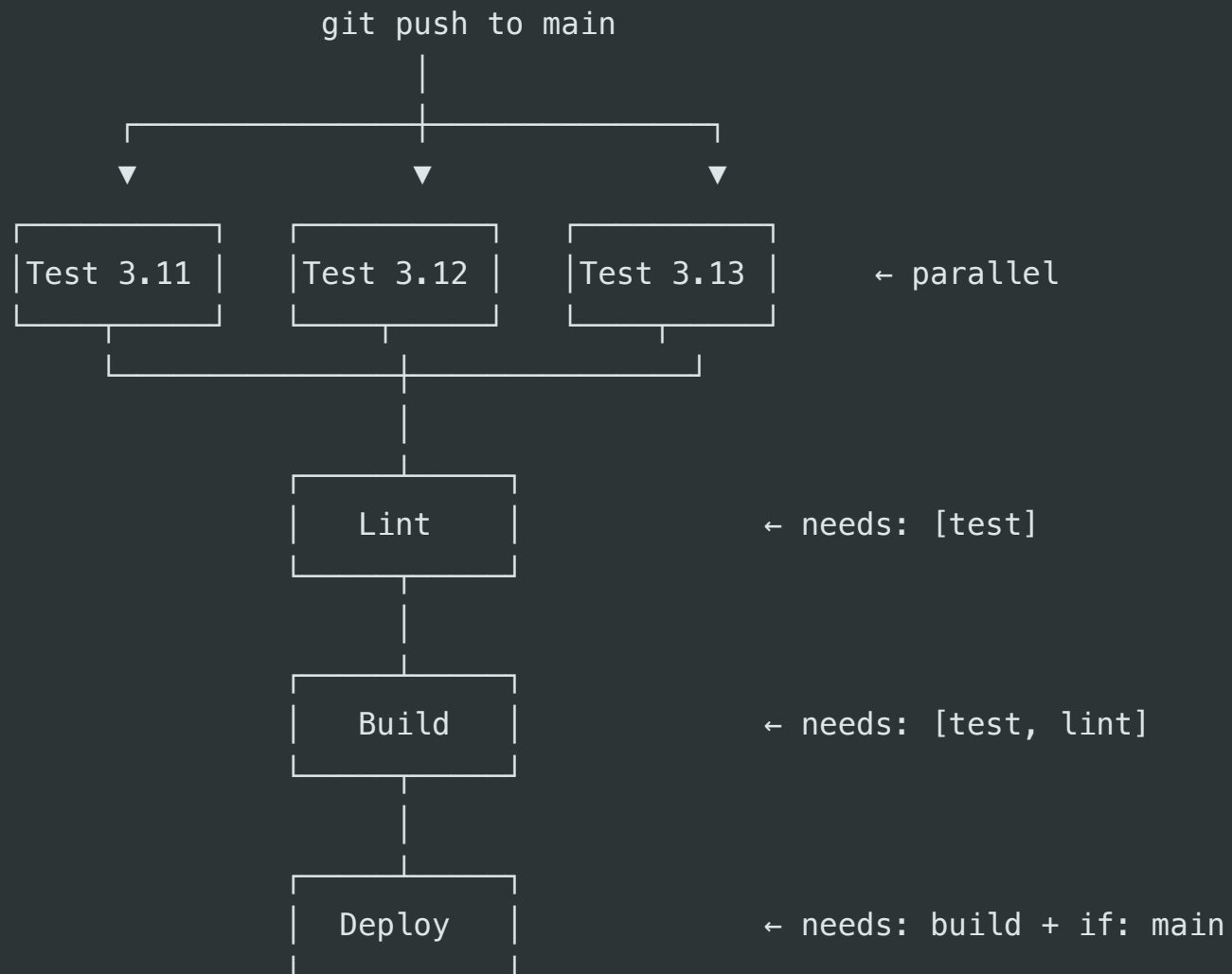
- Test  Lint  Build 
- Deploy  **RUNS!**

"Deploying now."

```
deploy:
  if: github.ref == 'refs/heads/main'
```

The feature branch is the rehearsal. Main is the performance.

The Full Pipeline



Exercise 2 & 3

Build the full pipeline — back to your laptops!

Exercise 2: Matrix + Linting

1. Replace your workflow with `exercises/02-matrix-and-lint.yml`
2. Push → watch **5 jobs** run in parallel (4 test + 1 lint)
3. **Break the linter**: add `import os` to top of `calculator.py`
4. Watch tests **pass** but lint **fail** — read the error
5. Remove the unused import → push → all green

Tests answer: "Does the code **work**?"

Linting answers: "Is the code **clean**?"

You need both.

Exercise 3: Full CI/CD Pipeline

1. Create a branch: `my-feature` from `main`
2. Replace workflow with `exercises/03-full-pipeline.yml`
3. Commit to `my-feature` → deploy is **skipped**
4. Open a Pull Request to main → checks run on the PR
5. Merge → deploy runs!

Feature branch:

Test ✓
Lint ✓
Build ✓
Deploy  SKIPPED

After merge to main:

Test ✓
Lint ✓
Build ✓
Deploy  RUNS!

You've just built a real CI/CD pipeline!

Best Practices

The rules that separate student projects from production

Best Practices

1. Keep workflows focused

One workflow, one purpose.

2. Use specific triggers

Don't run on every push to every branch.

3. Cache dependencies

First run: 45s. Cached: 3s.

4. Don't commit secrets

Use **GitHub Secrets**. Bots scan every public push.

5. Use Marketplace actions

20,000+ pre-built actions available.

6. Pin action versions

`@v6` for student projects. SHA for production.

Security: Pin Your Actions

```
# Student project – convenient:  
uses: actions/checkout@v6  
  
# Production – secure (exact commit SHA):  
uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683 # v6.0.2
```

Real-world incident (2024)

The action `tj-actions/changed-files` was **compromised**. An attacker hijacked the `@v35` tag and injected malicious code. Every project using `@v35` ran the attacker's code. Projects pinned by SHA were **unaffected**.

Student project: tags are fine. Production with credentials: always pin by SHA.

Quiz Time!

Talk to your neighbor for 30 seconds, then answer together.

Quiz — Question 1

You're trying to be a CI purist. Which habit aligns?

- a) Deploying code on Fridays, just before leaving for the weekend
- b) Frequently integrating code, even if it's only 50% tested
- c) Integrating code changes multiple times a day, with automated testing
- d) Writing code for a month, then integrating it all at once

Quiz — Answer 1

c) Integrating multiple times a day, with automated testing

Why not b? Frequently integrating *without tests* isn't CI — it's just frequent pushing. The **automated testing** part is non-negotiable.

Quiz — Question 2

Continuous Deployment is like...?

- a) A carefully planned, once-a-year code release party
- b) An autopilot that lands the plane after verifying every parameter
- c) Sending a letter by snail mail
- d) Keeping your code always ready to deploy, whenever you feel like it

Quiz — Answer 2

b) An autopilot that lands after verifying every parameter

Note: Option d) is Continuous **Delivery**, not Deployment. Delivery = ready to go, human decides. Deployment = fully automated.

Quiz — Question 3

Your job needs a machine. Which setting picks the runner?

- a) `steps`
- b) `runs-on`
- c) `uses`
- d) `name`

Quiz — Answer 3

b) **runs-on**

```
jobs:  
  test:  
    runs-on: ubuntu-latest    # ← This picks the machine
```

Options: **ubuntu-latest**, **windows-latest**, **macos-latest**, or self-hosted runners.

Quiz — Question 4

The **needs** keyword means...?

- a) "This job is super needy and requires constant attention"
- b) "This job can only run if these other jobs are successful"
- c) "This job needs more coffee"
- d) "This job is optional"

Quiz — Answer 4

b) "This job can only run if these other jobs are successful"

```
deploy:  
  needs: [test, lint]    # Deploy waits for both to pass
```

Quiz — Question 5

Where do GitHub Actions workflows live?

- a) In a secret, hidden folder
- b) In a folder named `.github/workflows/`
- c) In the cloud
- d) In your dreams

Quiz — Answer 5

b) In `.github/workflows/`

You created files there three times today. This should be automatic by now.

Quiz — Question 6: Find the Bug!

What's wrong with this workflow?

```
name: CI
on: push
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Set up Python
        uses: actions/setup-python@v6
        with:
          python-version: "3.12"
      - name: Run tests
        run: pytest
```

Take 60 seconds. Talk to your neighbor.

Quiz — Answer 6

Missing **actions/checkout**!

The runner has Python but **no code**. Pytest fails with "no tests found."

```
steps:
  - uses: actions/checkout@v6      # ← This was missing!
  - uses: actions/setup-python@v6
    with:
      python-version: "3.12"
  - run: pytest
```

The machine starts **empty**. Always checkout first. Most common beginner mistake.

What You Learned Today

Exercise 1

Basic CI

Tests on push

Red/green loop

Exercise 2

Matrix testing

Code linting

Caching

Exercise 3

Full pipeline

Artifacts

Branch protection

You went from **no automation** to a **full CI/CD pipeline** in 90 minutes.

Now Go Automate All the Things!

<https://github.com/navneetpandey/ci-cd-lab-2026>

Questions? Reach out anytime.

